

Problem Title

Median of Two Sorted Arrays

Difficulty Level: Hard

Problem Description (Including Scenario)

This problem was frequently asked by top product-based companies including **Google**, **Microsoft**, and **Amazon**.

You are given **two sorted arrays** `nums1` and `nums2` of sizes `m` and `n` respectively. Your task is to find the **median** of the two sorted arrays **without actually merging them**.

Important Requirement:

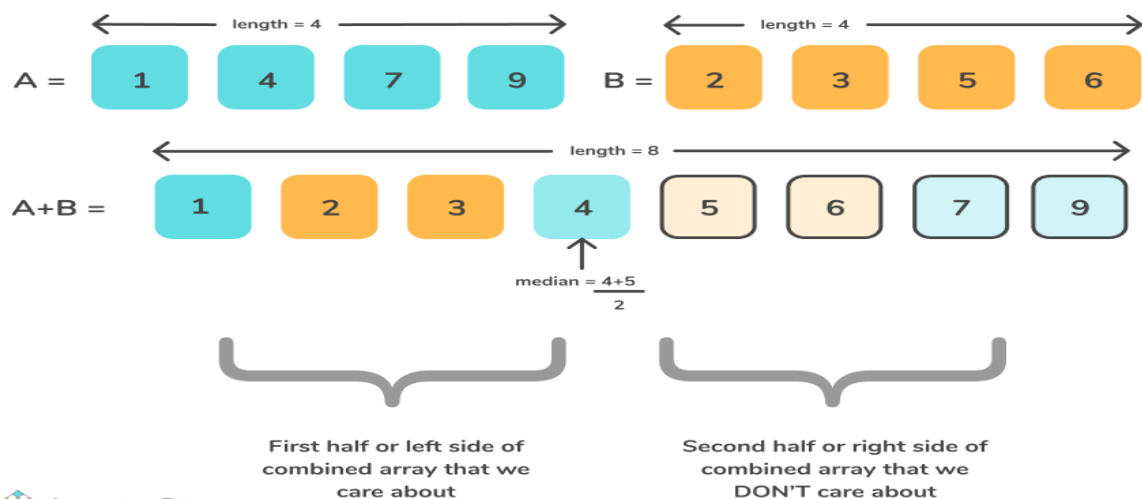
The overall time complexity must be:

$$O(\log(m + n))$$

This problem tests:

- Binary Search mastery
- Partitioning logic
- Edge case handling
- Mathematical reasoning

Visualized Example (Explanation)

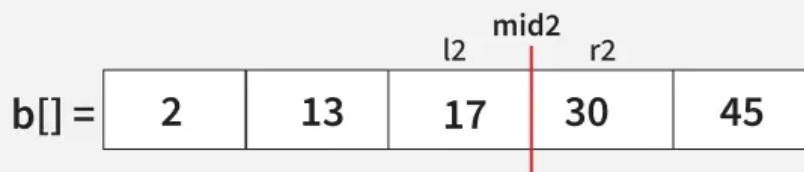
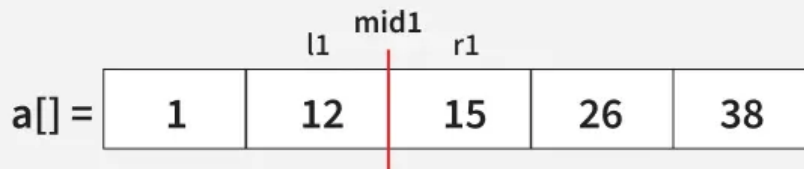


01
Step

Initialize low = 0 and high = 5

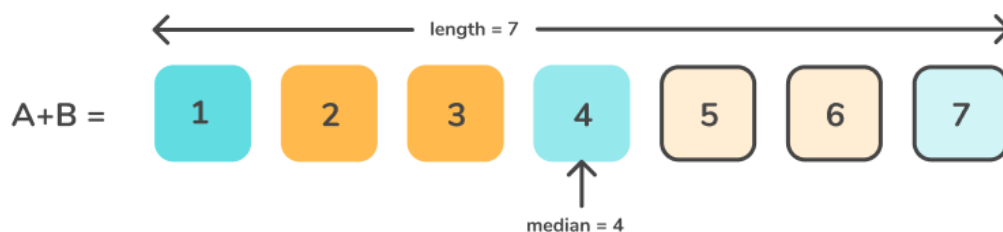
$\text{mid1} = (\text{low} + \text{high}) / 2 = 2$, $\text{mid2} = (\text{N} - \text{mid1}) = 3$

Since $\text{l2} > \text{r1}$, we have taken less elements from $a[]$, move $\text{low} = \text{mid1} + 1$



$\text{l1} \leq \text{r2}$ ✓
 $\text{l2} \leq \text{r1}$ ✗

Median of Two Sorted Arrays of same size using Binary Search



First half or left side of combined array that we care about

Second half or right side of combined array that we DON'T care about

Example 1

```
nums1 = [1, 3]  
nums2 = [2]
```

Merged (for understanding only):

[1, 2, 3]

Median = 2

✓ Output:

2.00000

Example 2

```
nums1 = [1, 2]  
nums2 = [3, 4]
```

Merged (for understanding only):

[1, 2, 3, 4]

Median = (2 + 3) / 2 = 2.5

✓ Output:

2.50000

Input Format

- Two sorted integer arrays `nums1` and `nums2`

```
nums1 = [1, 2]  
nums2 = [3, 4]
```

Output Format

- Return a **double value** representing the median

2.50000

Constraints

$0 \leq m \leq 1000$
 $0 \leq n \leq 1000$
 $1 \leq m + n \leq 2000$
 $-10^6 \leq \text{nums1}[i], \text{nums2}[i] \leq 10^6$

Topic Covered

- Arrays
- Binary Search
- Divide and Conquer
- Mathematical Logic
- Time & Space Optimization

Approaches

1. Brute Force Approach (Merge & Find Median)

Idea:

Merge both arrays into a single sorted array and then compute the median.

Algorithm:

1. Merge `nums1` and `nums2`
2. Sort the merged array
3. Calculate median

Time Complexity: $O(m + n)$

Space Complexity: $O(m + n)$

✗ Not allowed (violates problem constraint)

2. Optimized Approach – Binary Search Partition

Idea:

Apply **binary search on the smaller array** to find a partition such that:

- Left half contains exactly half of total elements
- All elements on left $\leq$ all elements on right

Key Insight:

$\text{LeftMax}(\text{nums1}, \text{nums2}) \leq \text{RightMin}(\text{nums1}, \text{nums2})$

Why Binary Search?

- Partition position can be searched efficiently
- Meets $O(\log(m+n))$ requirement

Time Complexity: $O(\log(\min(m, n)))$

Space Complexity: $O(1)$

 **Industry-standard solution**

Java Solution (Preferred Language)

```
class Solution {
    public double findMedianSortedArrays(int[] nums1, int[] nums2) {

        // Ensure nums1 is the smaller array
        if (nums1.length > nums2.length)
            return findMedianSortedArrays(nums2, nums1);

        int m = nums1.length;
        int n = nums2.length;
        int low = 0, high = m;

        while (low <= high) {
            int cut1 = (low + high) / 2;
            int cut2 = (m + n + 1) / 2 - cut1;

            int left1 = (cut1 == 0) ? Integer.MIN_VALUE : nums1[cut1 - 1];
            int left2 = (cut2 == 0) ? Integer.MIN_VALUE : nums2[cut2 - 1];
            int right1 = (cut1 == m) ? Integer.MAX_VALUE : nums1[cut1];
            int right2 = (cut2 == n) ? Integer.MAX_VALUE : nums2[cut2];

            if (left1 <= right2 && left2 <= right1) {
```

```
        if ((m + n) % 2 == 0) {
            return (Math.max(left1, left2) +
                    Math.min(right1, right2)) / 2.0;
        } else {
            return Math.max(left1, left2);
        }
    }
    else if (left1 > right2) {
        high = cut1 - 1;
    }
    else {
        low = cut1 + 1;
    }
}
return 0.0;
}
}
```

Practice Link (Live)

- **LeetCode**
<https://leetcode.com/problems/median-of-two-sorted-arrays/>
 - **GeeksforGeeks**
<https://www.geeksforgeeks.org/median-of-two-sorted-arrays/>
-

Video Solution Link

- **YouTube**
<https://www.youtube.com/watch?v=LPFhl65R7ww>
-

Learning Outcome

After solving this problem, the learner will be able to:

- Apply binary search on arrays
- Understand partition-based logic
- Optimize time complexity rigorously
- Solve hard interview-level problems
- Write clean and scalable Java code